

Apex Banking Platform — Backend & System Architecture

Comprehensive Walkthrough for Fullstack & Frontend Developers

Tech Stack: Python Flask, PyMongo, MongoDB, React Vite • Security: JWT (HS256) + Salted Bcrypt

1. High-Level Architecture & End-to-End Request Flow

The Apex Banking backend is organized as a **Layered RESTful Micro-Modular Architecture**. When a client performs an operation (e.g., Sarah transfers \$1,000 to David), the request traverses the following lifecycle:

Step 1: Frontend API Dispatch: React UI calls `api.post('/transactions/transfer', {...})`. The central client attaches the JWT token from `localStorage` into the Authorization: `Bearer <token>` header.

Step 2: Flask Routing & Factory: The request hits port 5000. Flask routes it through the `transactions_bp` Blueprint registered inside `app/__init__.py`.

Step 3: Security & Expiration Guard: The `@token_required` decorator decodes the JWT using `HS256 + JWT_SECRET`, validates the 5-minute session timeout, checks if the account is suspended/frozen, and binds `request.current_user`.

Step 4: Role-Based Authorization: The `@role_required(['customer', 'manager'])` decorator validates user role permissions.

Step 5: Business Validation & Fees: The controller verifies source balance, checks the \$100 minimum balance for Savings accounts, applies the \$1.50 fee for Business accounts, and looks up the destination account.

Step 6: Atomic Database Operations: MongoDB updates both account balances atomically via `update_one`, logs double-entry transaction documents, and logs an immutable audit event in `db.audit_logs`.

Step 7: Response & UI Sync: Returns 200 OK with new balances and transaction ID. The React frontend updates state and displays the official PDF-ready transaction receipt.

2. Backend Directory & Component Architecture

File / Module	Layer	Core Purpose & Responsibilities
<code>backend/run.py</code>	Server Entry	Reads port, creates Flask app instance via <code>create_app()</code> , and launches HTTP server.
<code>app/__init__.py</code>	App Factory	Initializes CORS, initializes database connection, registers all Blueprints, seeds default data.
<code>app/config.py</code>	Configuration	Encapsulates environment variables: MongoDB URI, JWT secrets, interest rates (6%/9%), timeouts.
<code>app/database.py</code>	Data Layer	Manages Singleton MongoDB connection pool via PyMongo, initializes unique indexes, handles fallbacks.

app/middleware.py	Security Guard	JWT token generation and verification, @token_required and @role_required decorators, audit logging.
app/routes/auth.py	Auth Controller	Customer/Manager registration, Bcrypt salted password verification, and JWT session token generation.
app/routes/accounts.py	Accounts Controller	Opening Savings/Business accounts, generating unique 6-digit account numbers, beneficiary lookup.
app/routes/transactions.py	Transaction Engine	Deposits, withdrawals, self & inter-bank transfers with ■1.50 fees, and 12-row paginated history.
app/routes/manager.py	Manager Portal	Bank-wide reserves analytics, account status updates (Active/Frozen/Closed), audit logs, interest runs.
app/utils/seeder.py	Data Seeder	Seeds initial manager and customer profiles, demo balances, and sample transactions on startup.

3. Software Engineering & Object-Oriented Patterns

1. Singleton Pattern (Connection Pool in database.py)

Problem: Opening a new database connection on every request causes high latency, memory bloat, and socket exhaustion.
Solution: The Database class holds a single static `_db` and `_client`. Calling `Database.get_db()` reuses the live connection pool across all concurrent requests.

2. Application Factory Pattern (create_app() in app/__init__.py)

Concept: Instead of hardcoding a global Flask instance, the app is created inside a factory function.
Benefit: Allows dynamically instantiating the app with different configurations (e.g. `TestConfig` for automated unit tests vs `ProductionConfig` for deployment).

3. Decorator Pattern / Aspect-Oriented Programming (in middleware.py)

Concept: Custom Python decorators (`@token_required`, `@role_required`) wrap route functions.
Benefit: Decouples cross-cutting security, JWT validation, and RBAC authorization from core business logic (DRY principle).

4. Encapsulation & Configuration Management (in config.py)

Concept: Sensitive parameters (JWT keys, interest percentages, minimum balance thresholds, database URIs) are encapsulated in the `Config` class and loaded from environment variables (`.env`).

4. MongoDB Collections & Indexing Architecture

Collection	Key Fields & Types	Database Indexes & Constraints
users	<code>_id</code> , <code>name</code> , <code>email</code> , <code>password_hash</code> , <code>role</code> , <code>phone</code> , <code>status</code> , <code>created_at</code>	Unique Index: <code>email</code> (prevents duplicate accounts)
accounts	<code>_id</code> , <code>account_number</code> , <code>user_id</code> , <code>owner_name</code> , <code>owner_email</code> , <code>account_type</code> , <code>balance</code> , <code>interest_rate</code> , <code>minimum_balance</code> , <code>transaction_fee</code> , <code>status</code>	Unique Index: <code>account_number</code> Index: <code>user_id</code>
transactions	<code>_id</code> , <code>transaction_id</code> , <code>account_number</code> , <code>user_id</code> , <code>type</code> , <code>amount</code> , <code>fee</code> , <code>balance_after</code> , <code>reference</code> , <code>timestamp</code>	Unique Index: <code>transaction_id</code> Compound Index: <code>timestamp</code> (DESC) for fast pagination
audit_logs	<code>_id</code> , <code>action</code> , <code>details</code> (JSON), <code>user_id</code> , <code>user_email</code> , <code>role</code> , <code>ip_address</code> , <code>timestamp</code>	Index: <code>timestamp</code> (DESC) for regulatory compliance audit queries

5. Core Banking Business Rules & Policy Summary

Rule Domain	Specification & Policy Enforcement
Savings Account	6.0% APY interest. Requires ■100.00 minimum initial deposit and balance. Withdrawals dropping balance below ■100 are rejected. Zero transaction fees.
Business Account	9.0% APY interest. No minimum balance required. Flat ■1.50 fee automatically charged per withdrawal and outgoing fund transfer.

Account Statuses	Active (Full operations), Frozen (All debits and credits blocked due to compliance/dispute), Closed (Deactivated). Managed strictly by Manager role.
Session Timeout	Customer: Strict 5-minute wall-clock session encoded in JWT. Topbar countdown alerts at ≤20s. Manager: 24-hour administrative session.
Audit Trail	Every state change (logins, transfers, status modifications) writes an immutable record with IP address, user email, and JSON payload.

6. Client Interview Cheat Sheet — Top Questions & Answers

Q1: Can you walk me through the backend architecture of this banking project?

The backend is built with Python Flask using a modular Blueprint REST architecture connected to MongoDB via PyMongo. We implemented the Application Factory pattern for flexibility, a Singleton connection pool for MongoDB, and custom middleware decorators for JWT authentication and Role-Based Access Control (RBAC). It powers multi-tier bank accounts, atomic fund transfers with fee calculations, a 5-minute customer session timeout, and an immutable compliance audit trail.

Q2: How is authentication, password hashing, and session management implemented?

Passwords are encrypted using Bcrypt with 12 salt rounds before database storage. Upon login, we issue signed JSON Web Tokens (HS256) containing the user's role and expiration. Customers have a strict 5-minute session lifetime, while managers receive 24-hour access. On every request, our `@token_required` decorator extracts the Bearer token, validates the signature, checks for expiration, and verifies the user is not suspended.

Q3: How is MongoDB structured and how do you ensure data integrity?

We use 4 primary collections: users, accounts, transactions, and audit_logs. We enforce unique indexes on emails, account numbers, and transaction IDs to guarantee data consistency. For fund transfers, we perform atomic updates using MongoDB's operator to ensure balances remain strictly synchronized, followed by creating double-entry ledger records and audit entries.

Q4: What specific banking rules did you implement?

We implemented two specialized account types: Savings Accounts (6.0% APY, \$100 minimum balance rule) and Business Accounts (9.0% APY, \$1.50 per-transaction fee). We also built manager controls allowing administrators to update account statuses between Active, Frozen, and Closed, along with an automatic annual interest calculation engine.

Q5: How does the React frontend communicate with this backend?

The frontend uses a centralized API service that automatically attaches the JWT Bearer token to all outgoing requests. If a customer session expires (401 response), the frontend catches it globally, displays a slide-over expiration warning, and safely redirects the user to login.